

Yarmouk University
Hijjawi Faculty for Engineering Technology
Department of Biomedical Systems and
Informatics Engineering



Motif finding with Gibbs sampling

Student Name	Student Number
1. Akram Ayman Abu-shmais	2015978004
2. Ghadeer Ghazi Migdady	2015978021

Supervisor
Ahmad Al-Omari

31/10/2020

Acknowledgement

We thankful for GOD to finish our project and come to this stage of our study, we thanks for our parents to support us to avoid our hard time, failures and for encourage us to solve any problem that we faced, we thanks our doctors and for colleagues helping us in solving some problems we faced in writing our CODE, and finally we thanks our grate supervisor Ahmad Al-omari for supporting and followings us to complete till the end by giving us some papers, books, hints, YouTube videos, and some web pages that we used helped us to get the project that between our hands right now.

Abstract

This project is talking about Gibbs sampler method, which is a statistical method that is built based on the probability of nucleotide position. The method depends on specific number of nucleotides from the original sequence to find motifs, which are recurring patterns in DNA that presumed to have a biological function. We used C++ language in our project. In genetics, a sequence motif is a nucleotide or amino-acid sequence pattern that is widespread and has, or is conjectured to have, a biological significance. For proteins, a sequence motif is distinguished from a structural motif, a motif formed by the three-dimensional arrangement of amino acids which may or may not be adjacent. When motif sequence appears in the exon of gene, it may encode the "structural motif " of a protein. We validated our result analytically and the accuracy and the speed were high as shown in the results section.

Table of content

ACKNOWLEDGEMENT	1
ABSTRACT	2
CHAPTER1	6
INTRODUCTION.....	6
1.1 THE MOTIF	6
1.2 GIBBS SAMPLER METHOD	8
1.3 GIBBS SAMPLER WITH MOTIF FINDING	9
CHAPTER2	11
METHODS.....	11
2.1 GIBBS SAMPLER STEPS TO GET THE MOTIF	11
2.1.1 Initialization	11
2.1.2 Profile matrix	11
2.1.3 Predictive update step	12
2.1.4 Sampling step.....	13
2.1.5 Normalization	14
2.1.6 chose random sample after the normalization	14
CHAPTER3	17
RESULTS AND DISCUSSIONS	17
CHAPTER4	22
CONCLUSIONS	22
REFERENCES	23
APPENDIX.....	25

Table of figures

Figure 1.1 :Example of motif	8
Figure 1.2 :Random selection of motifs	9
Figure 2.1 :Distribution of positions	15
Figure 2.2 :problem solution	15
Figure 2.3 :the position of first reject sequence.....	16
Figure 2.4 :The loop that stopped the algorithm	16
Figure 3.1 :input sequences	17
Figure 3.2 :Remainder of nucleotides	17
Figure 3.3 :profile matrix	18
Figure 3.4 :probability matrix.	18
Figure 3.5 :The weights of sequences.	18
Figure 3.6 :After the normalization.	18
Figure 3.7 :Distribution of normalizes values	19
Figure 3.8 :Random selection	19
Figure 3.9 :The found motif.	20
Figure 3.10 :logo for found motif	21

List of Tables

Table 2. 1: The profile matrix	12
Table 2. 2: The probability matrix	13
Table 2. 3: weights of sequences	13
Table 2. 4: After the normalization	14

CHAPTER 1

Introduction

Gipps sampler method is very useful way of simulating from distribution that are difficult to simulate from directly, based on algorithm that align protein sequences by means of predictive inference, using residue frequencies, this Gibbs sampling algorithm iteratively selects alignments in accordance with their conditional probabilities, the newly formed alignments in turn update an evolving residue frequency model, when equilibrium is reached the most probable alignment can be identified, If a detectable pattern is present, generally convergence is rapid. Effectively, the algorithm finds optimal local multiple alignments in linear time.

Our project usefulness appears when a sequence motif appears at the beginning of a gene, it may encode the "structural motif" of a protein that is a stereotypical element of the overall structure of the protein.

1.1The motif

In genetics, a sequence motif is a nucleotide or amino-acid sequence pattern that is widespread and has, or is conjectured to have, a biological significance. For proteins, a sequence motif is distinguished from a structural motif, a motif formed by the three-

dimensional arrangement of amino acids which may or may not be adjacent[1].

When a sequence motif appears in the exon of a gene, it may encode the "structural motif" of a protein; that is a stereotypical element of the overall structure of the protein. Outside of gene exons, there exist regulatory sequence motifs and motifs within the "junk", such as satellite DNA. Some of these are believed to affect the shape of nucleic acids, but this is only sometimes the case. For example, many DNA binding proteins that have affinity for specific DNA binding sites bind DNA in only its double-helical form. They are able to recognize motifs through contact with the double helix's major or minor groove[1].

Within a sequence or database of sequences, researchers search and find motifs using computer-based techniques of sequence analysis, such as BLAST. Such techniques belong to the discipline of bioinformatics[1].



Figure 1.1: Example of motif [2] .

1.2 Gibbs sampler method

It's component of matrices that we follow to get the motif. First one represents the frequency of the nucleotide, the second one represent the probability of the nucleotide, the third one represent the normalization that we choose randomly one of its probabilities to get the motif position.

A Gibbs sampler is a Markov chain Monte Carlo (MCMC) algorithm for obtaining a sequence of observations which are approximately from a specified multivariate probability distribution, when direct sampling is difficult. This sequence can be used to approximate the joint distribution to approximate the marginal distribution of one of the variables, or some subset of the variables or to compute an integral.

Gibbs sampling is commonly used as a means of statistical inference, especially Bayesian inference. It is a randomized algorithm (i.e. an algorithm that makes use of random numbers), and is an alternative to deterministic algorithms for statistical inference such as the expectation-maximization algorithm (EM). Gibbs sampling is applicable when the joint distribution is not known explicitly or is difficult to sample from directly, but the conditional distribution of each variable is known and is easy (or at least, easier) to sample from. The Gibbs sampling algorithm generates an instance from the distribution of each variable in turn, conditional on the current values of the other variables. It can be shown that the sequence of samples constitutes a Markov chain, and the stationary distribution of that Markov chain is just the sought-after joint distribution. [3]

1.3 Gibbs sampler with motif finding

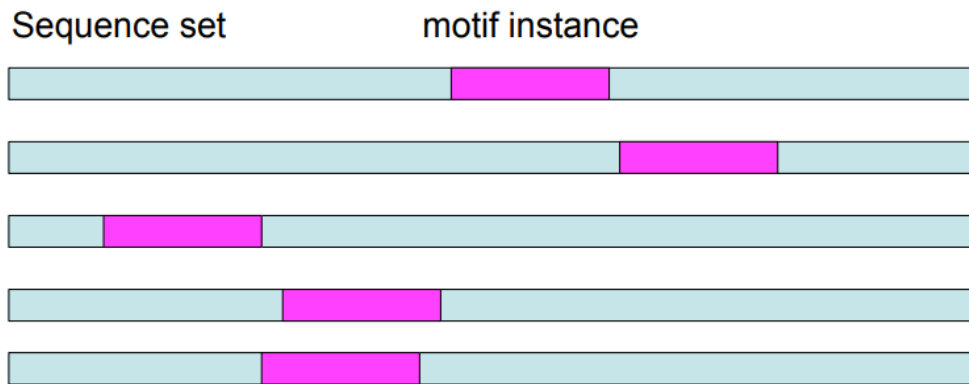


Figure 1.2: Random selection of motifs [4]

Gibbs sampling algorithm, which is a popular method for motif finding and has already been implemented by [5] and [6], is chosen for motif identification. Gibbs sampling is essentially a general stochastic strategy for determining the parameters of a statistical model relative to a given data set [7].

This strategy starts with certain initialization of parameter values and iteratively changes the value of one parameter at a time by assuming that the remaining parameters are correct and invoking Bayes Theorem until all parameters converge to stable (if not optimal) values. For changing the value of one parameter, Gibbs sampling employs a position weight matrix (PWM) as the motif model and selects the maximum likelihood value (highest score) for this specific parameter.

CHAPTER 2

Methods

2.1 GIBBS SAMPLER STEPS TO GET THE MOTIF

There are six steps in Gibbs sampler we follow to get the motif which are component of matrices that depends on statistical and computational operations.

2.1.1 Initialization

In this step we choose initial random position of motif, then we reject one sequence at each iteration and this one is the sequence that we find the motif position for it. We used the next sequences:

ACCATGACAG

GAGTATACCT

CATGCTTACT

CGGAATGCAT

*The range of random cut must be shorter than sequence length.

2.1.2 Profile matrix

It's the frequency of nucleotide by position from the random sequence that we choose in the previous step. In this matrix we calculate the background which is the frequency of nucleotide that remains out of random sequences.

Table 2. 1: The profile matrix : (Calculate the frequency of nucleotide in each position)

	Background	0	1	2	3	4	5	6
A	1	2	0	1	2	0	1	1
C	4	0	0	0	1	0	0	2
G	1	1	2	1	0	0	1	0
T	3	0	1	1	0	3	1	0

2.1.3 Predictive update step

Update profile matrix based on motif and background frequencies and pseudocounts which is a small number that we add to the calculation to avoid multiplying and divided by zero.

$$p_{z,n} = \frac{c_{z,n} + b_n}{N - 1 + B} \dots \dots \dots 1$$

Z: position

C: nucleotide frequency .

n: nucleotide.

b: pseudocounts .

N: nucleotides number.

B: Pseudocounts multiplied with N.

$$p_{0,n} = \frac{c_{0,n} + b_n}{\sum_{i \neq 1} (L_i - W) + B} \dots \dots \dots 2$$

A: nucleotide.

b: pseudocounts.

$\sum_{i \neq 1} (L_i - W)$: Summation of frequency of all background nucleotide.

B: Pseudocounts multiplied with N.

Table 2. 2: The probability matrix

	Background	0	1	2	3	4	5	6
d								
A	0.136	0.500	0.100	0.300	0.500	0.100	0.300	0.300
C	0.409	0.100	0.100	0.100	0.300	0.100	0.100	0.500
G	0.136	0.300	0.500	0.300	0.100	0.100	0.300	0.100
T	0.318	0.100	0.300	0.300	0.100	0.700	0.300	0.100

2.1.4 Sampling step

For each possible motif start position, we calculate the ratio of likelihood of next positions from motif vs. background.

$$p_n = \frac{p_{1,z} \cdot p_{2,z} \cdot p_{3,z} \cdot p_{4,z} \cdot p_{5,z} \cdot p_{6,z} \cdot p_{7,z}}{p_{0,z} \cdot p_{0,z} \cdot p_{0,z} \cdot p_{0,z} \cdot p_{0,z} \cdot p_{0,z} \cdot p_{0,z}} \dots \dots \dots 3$$

n: sequence number.

z: the nucleotide of motif in series .

Table 2. 3: weights of sequences

	Sequence 1	Sequence 2	Sequence 3	Sequence 4
Ai	8.554	0.081	.049	10.998

Sum of position with weight = 19.682

2.1.5 Normalization

After calculate the position with weight for each sequence ,We sum all this value and start a normalization process that each motif sequence weight divided by summation of all motif sequences weights .

Table 2. 4: After the normalization

	Sequence 1	Sequence 2	Sequence 3	Sequence 4
Ai	0.435	0.004	0.002	0.559

2.1.6 chose random sample after the normalization

In this step, the all probability of position in each sequences total one that the distribution of sample depend on the probability of positions.

The probability distribution is a mathematical function that provides the probabilities of occurrence of different possible outcomes in an experiment. [8][9]

In more technical terms, the probability distribution is a description of a random phenomenon in terms of the probabilities of events [10]. Examples of random phenomena can include the results of an experiment or survey. A probability distribution is specified in terms of an underlying sample space, which is the set of all possible outcomes of the random phenomenon being observed. The sample space may be the set of real numbers or a set of vectors, or it may be a list of non-numerical values.

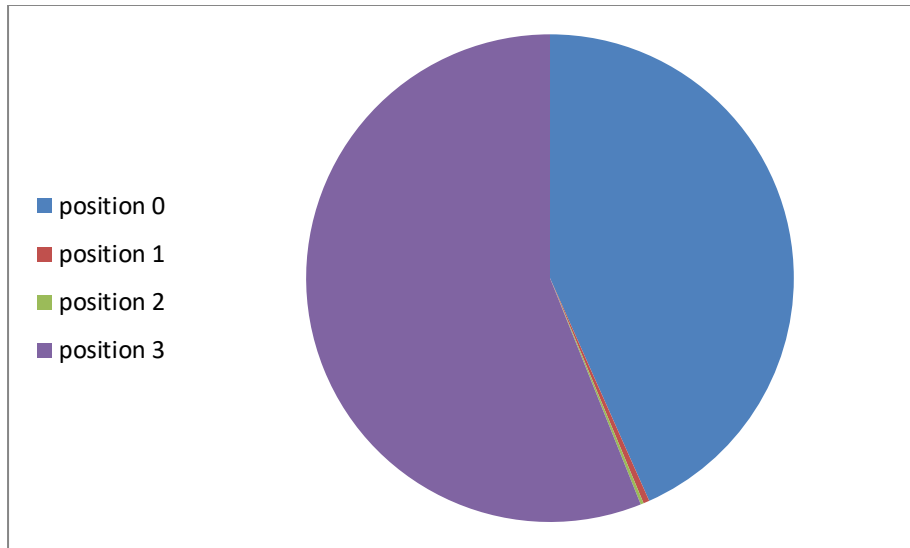


Figure 2.1 :Distribution of positions, we generate this figure using Microsoft Excel

In this step we faced a problem in how to find distribution, and we solve it by made the position probability distribution by allocated an array with size 1000, then we filled the array location by the position number where number of location = probability of that position*1000.

```
cout << "The normalization values: " << endl;
for (unsigned i = 0; i < (int)normalize.size(); i++)
{
    normalize[i] = weight[i] / sumofnormalization;
    cout << normalize[i] << " - ";
}

//change the normalization to the intger number and save in new vector name distrbution
vector <int>distribution;
for (int i = 0; i < (int)normalize.size(); i++)
{
    distribution.push_back(int(double(normalize[i] * 1000)));
}
```

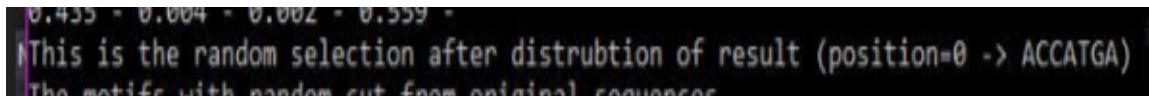
Fig2.2: problem solution

2.1.6.1 Choose random sample from distribution of positions.

We leave the first sequence and take all other sequences to find the their positions and then we leave the second sequences and take all other sequences to find the positions and so on until we find all positions.

In the best iteration, first position that selected is position zero

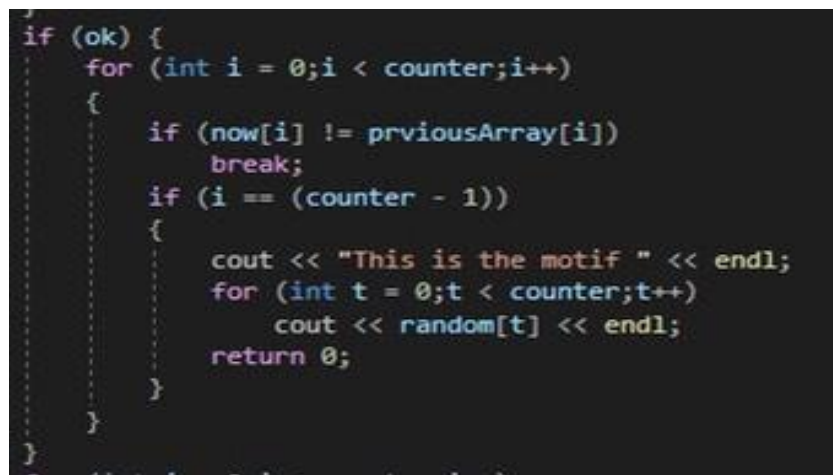
Like this:



```
0.435 - 0.004 - 0.002 - 0.559 -
This is the random selection after distrubtion of result (position=0 -> ACCATGA)
The motif with random cut from original sequences
```

Figure 2.3: the position of first reject sequence.

After much iteration, we choose motif positions and corresponding profile matrix. The code will be running, when the last array equal the current array with final positions.



```
if (ok) {
    for (int i = 0; i < counter; i++)
    {
        if (now[i] != previousArray[i])
            break;
        if (i == (counter - 1))
        {
            cout << "This is the motif " << endl;
            for (int t = 0; t < counter; t++)
                cout << random[t] << endl;
            return 0;
        }
    }
}
```

Figure 2. 4: The loop that stopped the algorithm .

CHAPTER 3

RESULTS AND DISCUSSIONS

After applied the steps that mentioned above using C++ code, we ran our project and we got the best motifs similarity.

We get the background (remainder of nucleotides after extracting motifs) using specific line of code that saving the reminder string in new integer string by location , and after applying this code we found background. Then, we counted the background frequency for each character in reminder string.

```
This project for applied Gibbs algorithm on sequences to find motifs...
The maximum sequences can be entered are 200 sequences.
All sequences muust equal the same length.
Please, enter the name of the existing sequences file: /users/dell/desktop/graduate project/c++/sequences.txt
Please, enter the length of motif nuclutide = 7
The initial sequences in strings
ACCATGACAG
GAGTATACCT
CATGCTTACT
CGGAATGCAT
```

figure 3.1 input sequences

```
The Remainder of nucleotides after extracting motiefs.
CAG
GCT
CAT
```

Figure 3.2 : Remainder of nucleotides

```

The matrix of nucleotide frequency by position.
A= 1 | 2 | 0 | 1 | 2 | 0 | 1 | 1 |
C= 4 | 0 | 0 | 0 | 1 | 0 | 0 | 2 |
G= 1 | 1 | 2 | 1 | 0 | 0 | 1 | 0 |
T= 3 | 0 | 1 | 1 | 0 | 3 | 1 | 0 |

```

Figure 3. 3: profile matrix

```

The matrix after applied probability equation:
A= 0.136 | 0.500 | 0.100 | 0.300 | 0.500 | 0.100 | 0.300 | 0.300 |
C= 0.409 | 0.100 | 0.100 | 0.100 | 0.300 | 0.100 | 0.100 | 0.500 |
G= 0.136 | 0.300 | 0.500 | 0.300 | 0.100 | 0.100 | 0.300 | 0.100 |
T= 0.318 | 0.100 | 0.300 | 0.300 | 0.100 | 0.700 | 0.300 | 0.100 |

```

Figure 3. 4 probability matrix.

```

8.554 = ACCATGA
0.081 = CCATGAC
0.049 = CATGACA
10.998 = ATGACAG

Sum of position with weigt = 19.682

```

Figure 3.5: The weights of sequences.

```

The Normalization values:
0.435 - 0.004 - 0.002 - 0.559 -

```

Figure 3.6: After the normalization.

Distrbution of positions

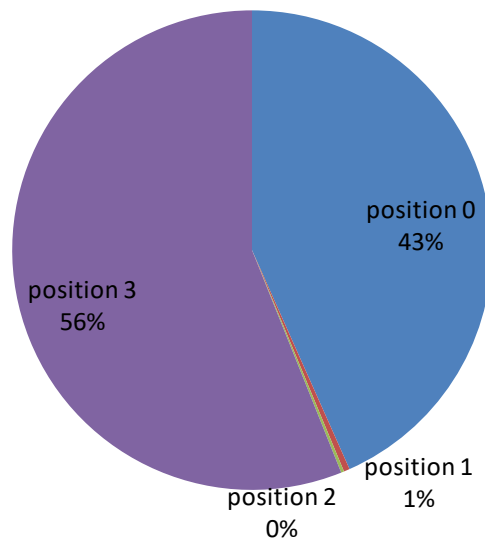


Figure 3.7: Distribution of normalizes values

We created this figure using Microsoft Excel

The distribution of position differs from the each other, that we found the largest distribution in position three and lowest distribution in position two ,but this process does not effect in selection a positions because its random selection .

```
This is the random selection after distrubtion of result (position=0 -> ACCATGA)
```

Figure3.8:Random selection

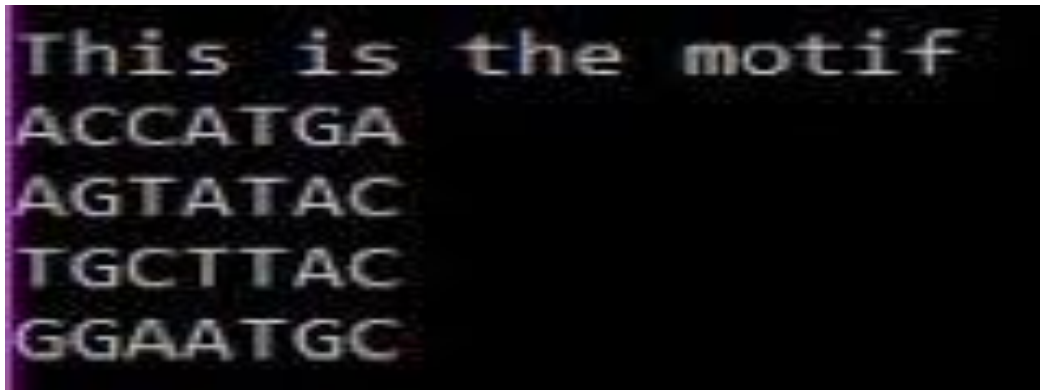


Figure 3.9: the found motif.

And in bioinformatics, we can use the sequence logo that is a graphical representation of the sequence conservation of nucleotides (in a strand of DNA/RNA) or amino acids (in protein sequences).[11] A sequence logo is created from a collection of aligned sequences and depicts the consensus sequence and diversity of the sequences. Sequence logos are frequently used to depict sequence characteristics such as protein-binding sites in DNA or functional units in proteins. A sequence logo consists of a stack of letters at each position. The relative sizes of the letters indicate their frequency in the sequences. The total height of the letters depicts the information content of the position, in bits that we created the logo sequences by web logo.

We used web logo to create logo for the best motif:

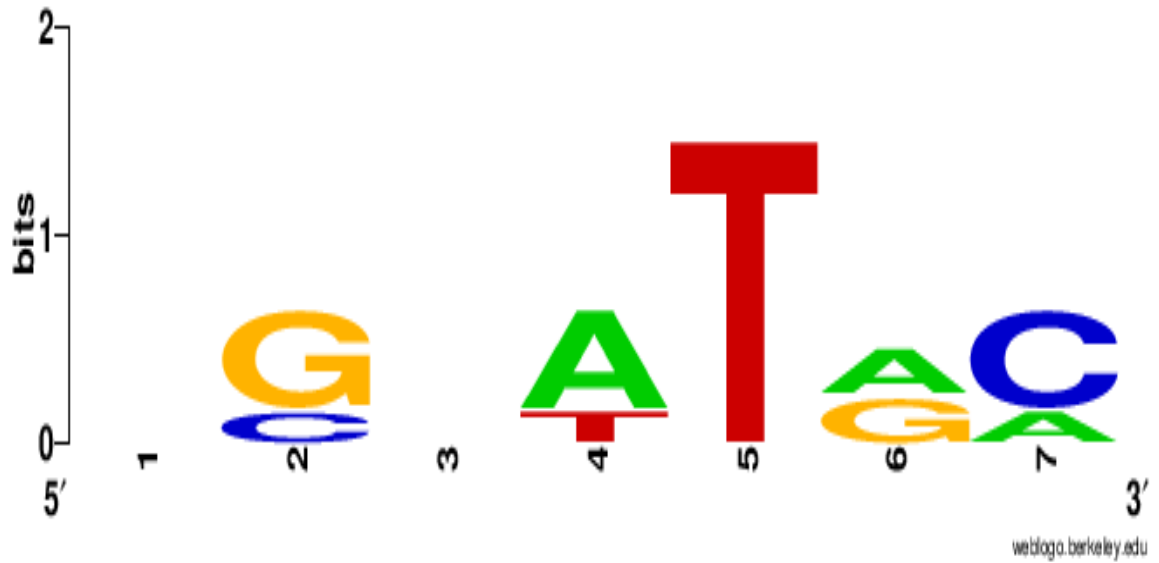


Figure 3. 10: logo for found motif. [12]

The accuracy of the output motifs was calculated from previous logo that shows the number of similarities between the nucleotide, the five positions have the highest similarities that have the same nucleotide of all sequences in the same position.

CHAPTER 4

Conclusions

We improve our level in C++ to be professional with it and we learned how to find motif using Gibbs sampler method which is a Markov chain Monte Carlo (MCMC), an important method that reduce time, effort, money, and increase the accuracy in finding motif. Gibbs sampler method depends on randomly statistical operations to get the motif sequence, and when we apply it on a randomly gene sequences we found an equal part of these sequences have higher similarity and these similar parts called motif.

During our work with Gibbs sampler method we discover one of its drawbacks which is that this method can't find the motif if it has the same nucleotide, on the other hand this method approved its ability to find an equal part of these sequences have higher similarity in general.

References

- [1]"Sequence_motif":https://en.wikipedia.org/wiki/Sequence_motif,[last accessed data 4 march 2020 at 09:40 PM].
- [2]Patrik D'haeseleer. "How does DNA sequence motif discovery work?"[last accessed data 21 February 2020 at 8:30 PM].
- [3]Gelman, Andrew and Carlin, John B and Stern, Hal S and Dunson, David B and Vehtari, Aki and Rubin, Donald B (2014). Bayesian data analysis. 2. FL: CRC press Boca Raton
- [4]C. Burge Lecture #9 Mar. 6, 2014 Modeling & Discovery of Sequence Motifs
- [5] Hughes, J.D., Estep, P.W., Tavazoie, S., and Church, G. M. Computational Identification of Cis-regulatory Elements Associated with Groups of Functionally Related Genes in *Saccharomyces Cerevisiae*. *Journal of Molecular Biology* 296, 1205-1214, 2000.
- [6]Liu, X., Brutlag, D.L., and Liu, J. S. BioProspector: Discovering Conserved DNA Motifs in Upstream Regulatory Regions of Co-expressed Genes. *Pacific Symposium on Biocomputing.*, 127-138, 2001.
- [7]Liu, J., Neuwald, A., Lawrence C. Bayesian Models for Multiple Local Sequence Alignment and Gibbs Sampling Strategies. *Journal of the American Statistical Association*, 90, 432, 1156–1170, 1995.
- [8]Everitt, Brian. (2006). *The Cambridge dictionary of statistics* (3rd ed.). Cambridge, UK: Cambridge University Press. ISBN 978-0-511-24688-3. OCLC 161828328
- [9] Ash, Robert B. (2008). "Basic probability theory (Dover ed.). Mineola, N.Y.: Dover Publications". pp. 66–69. ISBN 978-0-486-46628-6. OCLC 190785258..

- [10] Evans, Michael (Michael John) (2010). "Probability and statistics : the science of uncertainty". Rosenthal, Jeffrey S. (Jeffrey Seth) (2nd ed.). New York: W.H. Freeman and Co. p. 38. ISBN 978-1-4292-2462-8. OCLC 473463742
- [11] Schneider TD; Stephens RM (1990). "Sequence Logos: A New Way to Display Consensus Sequences". *Nucleic Acids Res.* 18 (20): 6097–6100. doi:10.1093/nar/18.20.6097. PMC 332411. PMID 2172928.
- [12] weblogo[Online] :<https://weblogo.berkeley.edu/logo.cgi>. [last accessed 7 February 2020].

Appendix

The code of our project:

```
#include <iostream>    // std::cin, std::cout
#include <fstream>      // std::ifstream
#include <string>
#include <time.h>
#include <cstdint>
#include <vector>
#include <iomanip>

using namespace std;
int main()
{
    char str[256];
    string seq[200]; // the initial all sequences
    string random[200]; // seq after cut
    string reminder[200]; // the reminder of sequences
    string x;
    int cut;
    int now[200]; // to save the work one
    int previousArray[200]; // save the previous position for all seq in file
    bool ok = false;
    cout << "This project for applied Gibbs algorithm on sequences to find motifs..." <<
endl;
    cout << "The maximum sequences can be entered are 200 sequences." << endl;
    cout << "All sequences must equal the same length." << endl;
    std::cout << "Please, enter the name of the existing sequences file: ";
    //users/dell/desktop/graduate project/c++/sequences.txt
    std::cin.get(str, 256);
    std::ifstream in(str); // open file
    ///////////////cheak if file open or not
    if (!in)
    {
        std::cerr << "Error: can't open file\n";
        return -1;
    }
    ///////////////////////////////////////////////////////////////////
    //initialization string each string have different line from file
    int counter = 0; // number of lines in file
    if (in.is_open())
    {
        while (getline(in, x))
        {
            seq[counter] = x;
            counter++;
        }
        counter = counter - 1;
    };
    in.close(); // close file
```

```

////////////////////////////////////
//print string have all sequences
cout << "Please, enter the length of motif nuclutide = ";
cin >> cut;
cout << "The initial sequences in strings " << endl;
for (int j = 0; seq[j] != "\0"; j++) //print all sequences in file(all lines)
{
    cout << seq[j] << endl;
}
////////////////////////////////////
//the new strings with random cut from original sequences
while (true)
{
    for (int Q = 0; Q < counter; Q++)
    {
        //for Q is number of string in file(sequences)
        cout << "The motifs with random cut from original sequences." <<
endl;

        for (int o = 0; o < counter; o++)
        {
            if (o == Q) continue;
            if (o < Q) {
                cout << random[o] << endl;
                continue;
            }

            if (seq[o].length() - cut - 1 <= 0) //the if to cheak
if randoms can applied on sequences or not
            {
                cout << "--The sequence equal or less than the cut
random--" << endl;

                return 1;
            }
            srand((unsigned int)time(NULL));
            int ran = (rand() % (seq[o].length() - cut - 1)); //randome
function give me from (0-(length-cut-1)) rand position cut
            random[o] = seq[o].substr(ran, cut); //save seq after cut from
random to fill number of cut

            //remover randome from all sequences
            reminder[o] = "";
            for (int i = 0; i < (int)seq[o].length(); i++) //
                if (i >= cut + ran || i < ran)
                    reminder[o] += seq[o][i];

            cout << random[o] << endl;
        };
    }
    //////////////////////////////////
    //the remainder of sequences after cutting
    cout << "The Remainder of nucleotides after extracting motiefs. " <<
endl;

    for (int t = 0; t < counter; t++) {
        if (t == Q) continue;
        cout << reminder[t] << endl;
    }
}

```

integer string by

printable characters

<< '\n';

```
////////////////////////////////////the code for save reminder string in new

//////////////////////////////// location and counting the frequency for each character
cout << "The background frequency of nucleotide." << endl;
string temp;
for (int i = 0; i < counter; i++)
{
    if (i == Q) continue;
    temp.append(reminder[i]);
}
size_t count[256];
int background[4] = { 0 };
std::fill_n(count, 256, 0);
for (unsigned int i = 0; i < temp.length(); i++)
{
    char t = temp[i];
    int ind = uint8_t(t);
    ++count[ind];
}; //character to index and adding value 1
/////define the array and save the background frequency in this array
unsigned int sumbackground = 0;
for (int index = 0; index < 256; index++)
{
    if (count[index] && isgraph(index)) // non-zero counts of

        {
            std::cout << char(index) << " = " << count[index]

            if (count[index] != 0) {
                if (char(index) == 'A')
                    background[0] = count[index];
                if (char(index) == 'C')
                    background[1] = count[index];
                if (char(index) == 'G')
                    background[2] = count[index];
                if (char(index) == 'T')
                    background[3] = count[index];
                sumbackground += count[index];
            }
        }
}
////////////////////////////////////
vector < vector<double>> matrix1(4, vector<double>(cut +
1)); //define 2d vector with 4 row and cut of motif +1 and save in first column th background
for (int j = 0; j < 4; j++)
{
    matrix1[j][0] = background[j];
}
int A = 0, C = 0, G = 0, T = 0;
for (int i = 0; i < cut; i++)
{
    for (int j = 0; j < counter; j++)
    {
```

```

        if (Q == j)continue;
        if (random[j][i] == 'A')
            A++;
        if (random[j][i] == 'C')
            C++;
        if (random[j][i] == 'G')
            G++;
        if (random[j][i] == 'T')
            T++;
        cout << random[j][i] << " ";
    }
    cout << "| A=" << A << " C=" << C << " G=" << G << "

T=" << T << endl;

    matrix1[0][i + 1] = A;
    matrix1[1][i + 1] = C;
    matrix1[2][i + 1] = G;
    matrix1[3][i + 1] = T;
    A = 0;C = 0;G = 0;T = 0;
}
//print the full vector after fill that by frequency for each character
A,C,G,T
cout << "The matrix of nucleotide frequency by position. " << endl;
for (unsigned int i = 0; i < matrix1.size(); i++)
{
    if (i == 0)cout << "A= ";
    if (i == 1)cout << "C= ";
    if (i == 2)cout << "G= ";
    if (i == 3)cout << "T= ";
    for (unsigned int j = 0; j < matrix1[i].size(); j++)
    {
        cout << matrix1[i][j] << " |";
    }
    cout << endl;
}
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
//the matrix will applied the main equation of gibbs algorithm with
threshold 0.5
cout << "The matrix after applied probability equation: " << endl;
for (unsigned int i = 0; i < matrix1.size(); i++)
{
    if (i == 0)cout << "A= ";
    if (i == 1)cout << "C= ";
    if (i == 2)cout << "G= ";
    if (i == 3)cout << "T= ";
    for (unsigned int j = 0; j < matrix1[i].size(); j++)
    {
        if (j == 0)
            matrix1[i][j] = ((matrix1[i][j] + 0.5) /
(sumbackground + 2));

        if (j != 0)
            matrix1[i][j] = ((matrix1[i][j] + 0.5) / (5));
        cout << setprecision(3) << fixed;
        cout << matrix1[i][j] << " |";
    }
}

```

```

    }
    cout << endl;
}
//size of one sequences in line number of charachter (nuclotide)in
line ///////////////

string y;
y = seq[Q];
char* p;
int u = 0;
p = &y[0];
for (unsigned int i = 0; i < seq->size(); i++)
{
    p[i]++;
    u++;
}
cout << "Number of nuclutide in each sequences= " << u << endl;
//////////////////////////calculate each position with wight and
saved in weight matrix

vector<double>weight(u - cut + 1);
vector<double>normalize(u - cut + 1);
double one = 0, two = 0, three = 0, four = 0;
int i = 0;
string q;
double sumofnormalization = 0;
while (i < u - cut + 1)
{
    q = seq[Q].substr(i, cut);
    for (int k = 0; k < cut; k++)
    {
        if (q[k] == 'A')
        {
            if (one != 0)
                one *= matrix1[0][k + 1] /
matrix1[0][0];
            else
                one = matrix1[0][k + 1] /
matrix1[0][0];
        }
        if (q[k] == 'C')
        {
            if (two != 0)
                two *= matrix1[1][k + 1] /
matrix1[1][0];
            else
                two = matrix1[1][k + 1] /
matrix1[1][0];
        }
        if (q[k] == 'G')
        {
            if (three != 0)
                three *= matrix1[2][k + 1] /
matrix1[2][0];

```

```

matrix1[2][0];
        else
            three = matrix1[2][k + 1] /
        }
        if (q[k] == 'T')
        {
            if (four != 0)
                four *= matrix1[3][k + 1] /
matrix1[3][0];
            else
                four = matrix1[3][k + 1] /
matrix1[3][0];
        }
        weight[i] = one * two * three * four;
    }
    cout << setprecision(3) << fixed;
    cout << weight[i] << " ";
    sumofnormalization += weight[i];
    cout << seq[Q].substr(i, cut) << endl;
    one = 0, two = 0, three = 0, four = 0;
    i++;
}
//normalization for number each position divided sum of wieght
cout << "sum of position with weigt = " << sumofnormalization <<
endl;

cout << "The normalization values: " << endl;
for (unsigned i = 0; i < (int)normalize.size(); i++)
{
    normalize[i] = weight[i] / sumofnormalization;
    cout << normalize[i] << " - ";
}
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
//change the normalization to the intger number and save in new
vector name distribution
vector<int>distribution;
for (int i = 0; i < (int)normalize.size(); i++)
{
    distrbution.push_back(int(double(normalize[i] * 1000)));
    //cout << distrbution[i] << " ";
}
cout << endl;
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
int size2 = 0; //sum of all value from 1000
for (int i = 0; i < (int)distrbution.size(); i++)
    size2 = size2 + distrbution[i];

vector<int >newdistrbution(size2);
int j = 0; int o = 0;
for (int i = 0; i < (int)distrbution.size(); i++)
{
    for (int k = 0; k < distrbution[i]; k++)
    {
        newdistrbution[j] = o;
    }
}

```

```

        j++;
    }
    o++;
}
srand(unsigned(time(NULL)));
int nor = rand() % newdistribution.size();
now[Q] = newdistribution[nor];
random[Q] = seq[Q].substr(newdistribution[nor], cut);
reminder[Q] = "";
for (int i = 0; i < (int)seq[Q].length(); i++)
    if (i >= cut + newdistribution[nor] || i < newdistribution[nor])
        reminder[Q] += seq[Q][i];
cout << "This is the random selection after distrubtion of result
(position=";

cout << newdistribution[nor] << " -> " << random[Q] << ")" << endl;
}
if (ok) {
    for (int i = 0; i < counter; i++)
    {
        if (now[i] != previousArray[i])
            break;
        if (i == (counter-1))
        {
            cout << "This is the motif " << endl;
            for (int t = 0; t < counter; t++)
                cout << random[t] << endl;
            return 0;
        }
    }
}
for (int i = 0; i < counter; i++)
    previousArray[i] = now[i];
ok = true;
}
return 0;
}

```